

CUDA Chapter 4 Notes: GPU Architecture, Warps, Scheduling, Divergence, and Occupancy

1 From CUDA Programming Model to GPU Hardware

Earlier chapters described CUDA programs using

grid $\rightarrow$ blocks $\rightarrow$ threads.

Chapter 4 explains how these logical CUDA objects are executed by the GPU hardware:

GPU $\rightarrow$ SMs $\rightarrow$ warps $\rightarrow$ threads.

The main goal is to understand how thread blocks are mapped to streaming multiprocessors (SMs), how blocks are divided into warps, and how GPU hardware hides latency by scheduling many resident warps.

2 GPU Architecture and Streaming Multiprocessors

2.1 High-level GPU organization

A CUDA-capable GPU contains many **streaming multiprocessors (SMs)**.

Each SM contains three broad categories of hardware resources:

- **Computation units:** arithmetic execution units that perform floating-point, integer, and other operations.
- **Control units:** hardware for instruction scheduling, issue, and execution control.
- **On-chip memory resources:** registers, shared memory, caches, and related structures.

All SMs can read from and write to the GPU's **global memory**. Global memory is large device memory shared by the whole GPU, while registers and shared memory are much faster on-chip resources associated with an SM.

A useful simplified view is therefore

$$\boxed{\text{GPU} = \text{many SMs} + \text{global memory}}$$

with

$$\boxed{\text{SM} = \text{compute} + \text{control} + \text{on-chip memory}}$$

2.2 CPU versus GPU design philosophy

A CPU is mainly optimized for **low latency** for a relatively small number of instruction streams. It therefore uses complex control logic, large caches, branch prediction, and powerful independent cores.

A GPU is mainly optimized for **high throughput**. Instead of making one thread extremely fast, it provides many execution resources and relies on massive thread-level parallelism.

Therefore:

CPU: minimize latency GPU: maximize throughput.

2.3 Global memory access

Memory allocated with `cudaMalloc` normally resides in device global memory:

```
float *d_A = nullptr;
cudaMalloc(&d_A, n * sizeof(float));
```

The pointer notation is important.

- `d_A` is the pointer variable. After `cudaMalloc`, it stores the device address of the allocated memory.
- In the declaration `float *d_A`, the `*` means that `d_A` has type `float*`.
- `*d_A` means **dereference the pointer**: access the float value stored at the address contained in `d_A`.
- `&d_A` is the address of the pointer variable itself. Its type is conceptually `float**`.

This is why `cudaMalloc` receives `&d_A`: the function needs to write the newly allocated device address into the pointer variable `d_A`.

2.4 Load and store operations

When a kernel accesses global memory, the SM performs load/store operations:

```
float x = A[i];           // load
C[i] = x + B[i];          // load B[i], compute, then store C[i]
```

A simple vector addition performs little arithmetic compared with its memory traffic:

$$C[i] = A[i] + B[i].$$

It reads two values, performs one addition, and writes one value. Such a kernel can therefore be limited by memory bandwidth rather than arithmetic throughput.

3 Block Scheduling on SMs

3.1 One block is assigned to one SM

When a kernel is launched, the grid contains many thread blocks. The hardware assigns each block as a **complete unit** to one SM:

one block $\rightarrow$ one SM

A block is not split across multiple SMs.

However, the reverse statement is not true: **one SM can hold several blocks simultaneously** if enough hardware resources are available.

Keeping a complete block on one SM allows all threads in that block to share the same on-chip shared memory and to synchronize efficiently with `__syncthreads()`.

3.2 A first introduction to warps

Before discussing SM resource limits, one more hardware unit is needed.

A **warp** is the basic thread scheduling unit inside an SM. On current NVIDIA CUDA GPUs, a warp normally contains

32 threads

.

After a block is assigned to an SM, the block is divided into warps. The SM schedules and issues instructions at warp granularity rather than independently scheduling every CUDA thread one by one.

Warps will be discussed in detail later in this chapter. At this point, it is enough to know that the maximum number of resident warps is one of the resources that can limit how many blocks fit on an SM.

3.3 Resources that limit resident blocks

The number of blocks that can reside simultaneously on one SM is limited by resources such as

- maximum resident threads per SM;
- maximum resident warps per SM;
- maximum resident blocks per SM;
- registers used per thread;
- shared memory used per block.

Therefore, block residency is determined by both **block size** and **kernel resource usage**. For example, suppose

```
myKernel<<<1000, 256>>>(...);
```

This launch creates

$$1000 \times 256 = 256000$$

threads.

If a hypothetical GPU has 8 SMs and each SM can hold 4 such blocks, then at most

$$8 \times 4 = 32$$

blocks can be resident at one time. The other blocks wait. When a resident block finishes, its resources are released and another waiting block can be scheduled.

CUDA does not guarantee block execution order. Blocks in the same kernel should normally be independent.

4 Synchronization and Transparent Scalability

4.1 Block-level barrier synchronization

Threads in the same block can synchronize using

```
__syncthreads();
```

A barrier divides execution into phases. Every thread in the block must reach the same barrier before any thread continues beyond it.

A common pattern is:

```
__shared__ float tile[256];

int tid = threadIdx.x;
tile[tid] = A[blockIdx.x * blockDim.x + tid];

__syncthreads();

float x = tile[255 - tid];
```

The barrier guarantees that all writes to `tile` have completed before threads start reading values written by other threads.

The critical rule is:

all threads in a block must reach the same barrier

A barrier placed only in one branch of divergent control flow can cause invalid behavior.

4.2 Transparent scalability

Blocks in the same kernel do not normally synchronize with one another. This independence allows the same CUDA source code to run on GPUs with different numbers of SMs.

A smaller GPU may execute fewer blocks simultaneously, while a larger GPU may keep more blocks resident at the same time. The source code does not need to change; the main difference is performance.

This property is called **transparent scalability**.

The practical rule is:

same kernel source + different GPU resources $\Rightarrow$ same computation, different performance

5 Warps and Thread Linearization

5.1 Blocks are partitioned into warps

After a block is assigned to an SM, the hardware divides its threads into groups of 32.

For a one-dimensional block with 256 threads:

$$\frac{256}{32} = 8$$

warps are created.

The warp assignment is

Warp 0 : threads 0–31,
Warp 1 : threads 32–63,
...
Warp 7 : threads 224–255.

If a block contains 48 threads, the hardware still allocates two warps. The second warp contains only 16 active threads, while the remaining 16 lanes are unused.

This is one reason block sizes are often chosen as multiples of 32.

5.2 Two-dimensional blocks use row-major linearization

For a two-dimensional block, CUDA first converts the two-dimensional thread coordinate into a linear order.

The x coordinate changes fastest, so the order follows the same row-major idea used by C/C++ arrays:

$$\text{linear_tid} = \text{threadIdx.y} \times \text{blockDim.x} + \text{threadIdx.x}$$

The resulting linear thread indices are then divided into groups of 32.

For a three-dimensional block:

$$\begin{aligned} \text{linear_tid} = & \text{threadIdx.z} \times \text{blockDim.y} \times \text{blockDim.x} \\ & + \text{threadIdx.y} \times \text{blockDim.x} + \text{threadIdx.x}. \end{aligned}$$

5.3 Warp ID and lane ID

Once the linear thread index is known,

$$\text{warp_id} = \left\lfloor \frac{\text{linear_tid}}{32} \right\rfloor$$

and

$$\text{lane_id} = \text{linear_tid} \bmod 32.$$

The **warp ID** tells us which warp inside the block contains the thread.

The **lane ID** tells us the thread's position inside that warp, from 0 to 31.

Consider a 16×16 block. Then

$$\text{linear_tid} = 16y + x.$$

For thread

$$(\text{threadIdx.x}, \text{threadIdx.y}) = (5, 3),$$

we obtain

$$\text{linear_tid} = 3 \times 16 + 5 = 53.$$

Therefore,

$$\text{warp_id} = \left\lfloor \frac{53}{32} \right\rfloor = 1,$$

and

$$\text{lane_id} = 53 \bmod 32 = 21.$$

So this thread belongs to **Warp 1** and occupies **lane 21** within that warp.

Two useful boundary examples are

$$(15, 1) \rightarrow \text{linear_tid} = 31 \rightarrow (\text{warp} = 0, \text{lane} = 31),$$

and

$$(0, 2) \rightarrow \text{linear_tid} = 32 \rightarrow (\text{warp} = 1, \text{lane} = 0).$$

5.4 Thread versus lane

A CUDA **thread** is a logical execution instance of the kernel.

A **lane** is the position, 0–31, occupied by that thread within its warp when the warp instruction is executed.

Therefore, lane 5 does not identify one unique thread in the whole block. Every warp has its own lane 5.

For example:

thread 5 $\rightarrow$ Warp 0, lane 5,

while

thread 37 $\rightarrow$ Warp 1, lane 5.

6 SIMT Execution, Fetch, Dispatch, and the von Neumann Model

6.1 Classical von Neumann organization

A simplified von Neumann processor contains four main parts:

- **Memory:** stores instructions and data.
- **Input/Output:** communicates with external devices.
- **Control unit:** manages the instruction stream.
- **Processing unit:** performs arithmetic and logical operations.

The control unit includes structures such as

- **PC (Program Counter):** stores the address of the next instruction.
- **IR (Instruction Register):** stores the instruction currently being processed.
- instruction decode and control logic.

The processing unit includes

- **ALU:** performs arithmetic and logic operations;
- **register file:** stores fast temporary operands and results.

6.2 Fetch, decode, and dispatch

A simplified instruction cycle is

fetch $\rightarrow$ decode $\rightarrow$ dispatch/execute.

Fetch means using the PC to obtain the next instruction from memory and placing it into the IR.

Decode means determining what operation the instruction requests and what operands or resources it needs.

Dispatch means sending the decoded operation to the appropriate execution hardware.

6.3 How the GPU extends this idea

The GPU keeps the same basic idea of control plus processing, but the control hardware is shared across many execution lanes.

Conceptually, one warp instruction is fetched and dispatched to the active lanes of a warp. Each lane executes the same instruction but uses the register values associated with its own CUDA thread.

This produces CUDA's **SIMT** model:

Single Instruction, Multiple Threads

The hardware idea is related to SIMD, but CUDA exposes independent logical threads to the programmer.

For example, a warp may execute

```
C[i] = A[i] + B[i];
```

for 32 different values of i . The instruction is shared, but each thread operates on different data.

This organization reduces the amount of independent control hardware required for every arithmetic lane and allows more hardware resources to be devoted to throughput.

7 Control Divergence

7.1 Divergence in an if-else statement

SIMT execution is most efficient when all threads in a warp follow the same control-flow path.

Consider

```
if (threadIdx.x < 24) {  
    A;  
}  
else {  
    B;  
}  
  
C;
```

Assume one warp contains threads 0–31.

Threads 0–23 satisfy the condition, while threads 24–31 do not.

The warp cannot execute instructions from paths A and B simultaneously because the warp issues one instruction stream at a time.

Conceptually, execution proceeds as follows:

Pass 1: execute A	lanes 0–23 active, lanes 24–31 inactive,
Pass 2: execute B	lanes 24–31 active, lanes 0–23 inactive,
Reconverge: execute C	all lanes active again.

This behavior is called **control divergence**.

The important point is not that an `if` statement is inherently slow. Divergence occurs only when threads **inside the same warp** make different branch decisions.

7.2 Relation between lanes and divergent threads

In Warp 0, logical threads 0–31 correspond to lane IDs 0–31.

Therefore, in the example above:

threads 0–23 $\leftrightarrow$ lanes 0–23

execute path *A*, and

threads 24–31 $\leftrightarrow$ lanes 24–31

execute path *B*.

The inactive lanes are temporarily masked while the other branch is executing.

7.3 Divergence in loops

Divergence can also occur when different threads execute different numbers of loop iterations:

```
int N = a[threadIdx.x];  
  
for (int i = 0; i < N; ++i) {  
    A;  
}
```

If threads in one warp have different values of *N*, some threads finish earlier than others.

The warp must continue executing the loop until the thread with the largest required iteration count has finished. Threads that have already completed their work become inactive during the remaining iterations.

Therefore, irregular per-thread workloads can reduce warp efficiency.

7.4 Boundary checks

Boundary checks can also cause divergence:

```
int i = blockIdx.x * blockDim.x + threadIdx.x;  
  
if (i < n) {  
    C[i] = A[i] + B[i];  
}
```

If *n* is not a multiple of the block size, only the final warp or final few warps are usually affected. For large arrays, this boundary divergence is normally acceptable because it represents a small fraction of the total work.

8 Warp Scheduling and Latency Hiding

8.1 Resident warps

An SM usually holds more threads and warps than it can execute at one instant.

The chapter uses the A100 as an example: one SM can have up to 2048 resident threads, corresponding to

$$\frac{2048}{32} = 64$$

resident warps.

These threads are **resident**: their execution state and required resources are assigned to the SM, but they are not all performing arithmetic simultaneously.

A CUDA thread is therefore not permanently bound to one fixed CUDA core.

8.2 Why many resident warps are useful

Global memory operations can take many clock cycles.

Suppose Warp 0 executes

```
float x = A[i];
```

and the requested data is not immediately available. Warp 0 cannot execute a later instruction that depends on x .

Instead of leaving the execution units idle, the SM's warp scheduler can select another ready resident warp:

Warp 0: waiting for memory

Warp 1: ready → execute Warp 1

Warp 2: ready → execute Warp 2.

When Warp 0's data becomes available, it becomes ready again and can return to the scheduling pool.

This mechanism is called

latency hiding

or latency tolerance.

The GPU does not necessarily make the memory access itself faster. Instead, it performs useful work from other warps while one warp is waiting.

8.3 Low-overhead warp switching

Traditional CPU thread context switching may require saving and restoring execution state.

GPU resident warps already have their execution state stored in SM resources such as the register file. Therefore, switching instruction issue from one resident warp to another is extremely cheap compared with a traditional software thread context switch.

This is often described as **zero-overhead warp scheduling**.

9 Resource Partitioning and Occupancy

9.1 Definition of occupancy

Occupancy describes how many warps are resident on an SM relative to the hardware maximum:

$$\text{Occupancy} = \frac{\text{resident warps per SM}}{\text{maximum resident warps per SM}}$$

Because a warp contains 32 threads, thread counts can often be used equivalently:

$$\text{Occupancy} \approx \frac{\text{resident threads per SM}}{\text{maximum resident threads per SM}}.$$

Higher occupancy can provide more candidate warps for latency hiding, but

higher occupancy ≠ automatically higher performance.

9.2 Resources that limit occupancy

The number of resident blocks is approximately limited by the minimum of several resource constraints:

$$\text{resident blocks/SM} = \min(\text{thread limit,} \\ \text{warp limit,} \\ \text{block limit,} \\ \text{register limit,} \\ \text{shared-memory limit}).$$

A complete block can become resident only if enough resources are available for the whole block.

9.3 Block size can reduce occupancy

Using the chapter's A100 example with a maximum of 2048 resident threads and 32 resident blocks per SM:

For 1024 threads per block,

$$\frac{2048}{1024} = 2$$

blocks can reside from the thread-count limit.

For 256 threads per block,

$$\frac{2048}{256} = 8$$

blocks can reside from the thread-count limit.

However, if the block size is only 32 threads, filling 2048 thread slots would require

$$\frac{2048}{32} = 64$$

blocks, but the SM supports only 32 resident blocks. Therefore only

$$32 \times 32 = 1024$$

threads are resident, giving

$$\text{Occupancy} = \frac{1024}{2048} = 50\%.$$

Thus, a block can be **too small** if another hardware limit is reached before all thread slots are filled.

9.4 Resource fragmentation

Suppose a block has 768 threads. The SM can fit two blocks:

$$2 \times 768 = 1536$$

threads.

A third block would require

$$3 \times 768 = 2304 > 2048.$$

Therefore 512 thread slots remain unused, and occupancy is

$$\frac{1536}{2048} = 75\%.$$

The remaining slots cannot be used because CUDA admits resources at block granularity.

9.5 Registers and occupancy

Registers are another major occupancy constraint.

Using the chapter's simplified A100 example with 65,536 registers per SM, full occupancy with 2048 threads would allow, on average,

$$\frac{65536}{2048} = 32$$

registers per thread.

If a kernel requires 64 registers per thread, the register-file limit allows only

$$\frac{65536}{64} = 1024$$

resident threads, so the occupancy cannot exceed approximately

$$\frac{1024}{2048} = 50\%.$$

A small increase in register usage can therefore reduce the number of complete resident blocks. This sudden occupancy drop is sometimes called a **performance cliff**.

If register demand becomes too high, the compiler may perform **register spilling**, storing some values in slower memory. Reducing register usage may improve occupancy, but excessive spilling can hurt performance.

The practical goal is not simply to maximize occupancy, but to balance

occupancy against per-thread efficiency and data reuse.

10 Querying CUDA Device Properties

Different GPUs provide different numbers of SMs and different limits for threads, blocks, registers, shared memory, and grid dimensions.

CUDA therefore provides runtime APIs for querying the actual hardware.

10.1 Count CUDA devices

```
int devCount = 0;
cudaGetDeviceCount(&devCount);
```

CUDA devices are indexed from 0 to `devCount - 1`.

10.2 Read device properties

```
cudaDeviceProp devProp;

for (int i = 0; i < devCount; ++i) {
    cudaGetDeviceProperties(&devProp, i);
}
```

Important fields include:

- `devProp.multiProcessorCount`: number of SMs;
- `devProp.maxThreadsPerBlock`: maximum threads in one block;
- `devProp.maxThreadsDim[0..2]`: maximum block dimensions;
- `devProp.maxGridSize[0..2]`: maximum grid dimensions;
- `devProp.warpSize`: hardware warp size;
- register and shared-memory capacity fields;
- `devProp.clockRate`: device clock information.

10.3 Compute capability

Compute capability identifies the architectural generation and supported CUDA hardware features.

It should be understood as

architecture/capability identifier

rather than a direct performance score.

Device queries are useful because the same CUDA source can run on different GPUs whose resource limits and performance characteristics are not identical.

11 Execution Hierarchy to Remember

The most important hierarchy in this chapter is

kernel → grid → blocks → SMs → warps → threads.

The key performance relationships are

block scheduling → SM resource usage,
 warp execution → SIMT efficiency,
 divergence → inactive lanes,
 many resident warps → latency hiding,
 register/shared-memory usage → occupancy.

These concepts form the foundation for later CUDA performance analysis and kernel optimization.